

Ejercicios de Repaso

Programacion · 9 ejercicios agrupados por bloque temático

RA1	UT2 - Introduccion a la programacion
RA3	UT2 - Estructuras de control
RA2	UT3 - Introduccion a POO
RA4	UT3 - Clases y objetos
RA5	UT10 - Ficheros con ArrayList
RA6	UT6, UT9 - Arrays y colecciones (Strings)
RA7	UT7 Excepciones + UT8 POO avanzada
RA8	Tema 11 - JDBC (Crear tabla + Insertar)
RA9	Tema 11 - JDBC (SELECT + UPDATE)

Los ejercicios están agrupados por bloque temático. Ficheros usa **ArrayList**. Colecciones se centra en **String**. JDBC usa la BD tienda con la tabla empresa(id, nombre, tipo), los mismos patrones que los ejemplos de clase. Las soluciones están al final.

Bloque 1 – UT2: Introducción a la programación y Estructuras de control

RA1

UT2 – Introducción a la programación

Escribe un programa que pida por teclado el **precio** de un producto (double) y el **número de unidades** (int). El programa debe calcular el importe total, aplicarle un **21 % de IVA** y mostrar por pantalla el precio final con un mensaje claro.

Usa Scanner para la entrada y variables del tipo adecuado.

Tu respuesta:

RA3

UT2 – Estructuras de control | ¿Qué se muestra por pantalla?

Analiza el siguiente código **sin ejecutarlo** y escribe la **salida exacta**, línea por línea, que aparece por pantalla:

```
int suma = 0;
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) {
        suma += i;
    } else {
        suma -= 1;
    }
    System.out.println("i=" + i + " suma=" + suma);
}
System.out.println("Total: " + suma);
```

Salida exacta:

Bloque 2 – UT3: Introducción a POO y Clases y objetos

RA2

UT3 – Introducción a la POO

Crea una clase **Producto** con dos atributos: nombre (String) y precio (double). Añade un método `mostrar()` que imprima los dos atributos.

En una clase con `main`, crea **dos** objetos `Producto`, asigna sus valores y llama a `mostrar()` en cada uno.

Tu respuesta:

RA4

UT3 – Clases y objetos

Amplía la clase **Producto** del ejercicio anterior:

- Haz los atributos **privados** y añade **constructor**, **getters** y **setters**.
- Sobrescribe `toString()` para devolver "Producto: nombre - X.XX €".
- Añade el método `aplicarDescuento(double pct)` que reste ese porcentaje al precio.

En el `main`: crea un `Producto` con el constructor, aplícale un 10 % de descuento y muéstralo con `System.out.println(p)`.

Tu respuesta:

Bloque 3 – UT10: Ficheros con ArrayList

RA5

UT10 – Ficheros con ArrayList (versión sencilla)

El fichero **Biblioteca.txt** tiene una línea por documento con este formato:

```
Titulo:Autor:Codigo:Anio:Estado:Edicion
```

Ejemplo de contenido real del fichero:

```
Cien Anios de Soledad:Garcia Marquez:9780307474728:1967:DISPONIBLE:1 edicion
National Geographic:Varios:12345678X:2020:PRESTADO:National Geographic
El Senor de los Anillos:Tolkien:9780618640157:1954:DISPONIBLE:2 edicion
...
```

Escribe un programa que **lea el fichero línea a línea** con `BufferedReader` y guarde el **título** (primer campo, separando por `:`) en un `ArrayList<String>`.

Al terminar de leer, recorre el `ArrayList` con un `for-each` y muestra: (1) cuántos títulos hay en total con `.size()` y (2) todos los títulos. No hay que ordenar ni escribir en ningún fichero.

Recuerda usar try-catch y cerrar el fichero.

Tu respuesta:

Bloque 4 – UT6/UT9: Colecciones de String

RA6

UT6, UT9 – Colecciones de String | Mejora el código para que funcione

El siguiente programa recorre un `ArrayList<String>` y debería mostrar en **MAYÚSCULAS** los nombres que empiezan por 'A', pero **no funciona**. Tiene **dos errores**: uno impide que el código compile y otro provoca una excepción en tiempo de ejecución. **Encuétralos y reescribe el código corregido**.

```
ArrayList<String> nombres = new ArrayList<>();
nombres.add("ana");
nombres.add("Pedro");
nombres.add("Alba");
nombres.add("luis");

for (int i = 0; i <= nombres.size(); i++) {
    String n = nombres.get(i);
    if (n.charAt(0) == "A") {
        System.out.println(n.toUpperCase());
    }
}
```

Errores encontrados y código corregido:

Bloque 5 – UT7 Excepciones + UT8 POO avanzada

RA7

UT7 Excepciones + UT8 POO avanzada

Crea una clase **abstracta** `Figura` con un método abstracto `double area()`. Crea dos clases hijas: `Circulo` (atributo `radio`) y `Rectangulo` (`base` y `altura`), cada una con su propio `area()`.

En el `main`, parte de este texto introducido por el usuario (simulado como `String`) para el `radio`:

```
String entrada = "hola"; // debería ser un numero
```

Convierte la entrada a `double` con `Double.parseDouble(entrada)` dentro de un `try-catch` que capture `NumberFormatException` y muestre *"Radio no válido"*. Si la conversión es correcta, crea un `Circulo` y muestra su área.

Tu respuesta:

Bloque 6 – Tema 11: Bases de datos con JDBC

RA8

Tema 11 – JDBC | Crear tabla e insertar datos

Trabaja sobre la base de datos MySQL **tienda**. Los datos de conexión son:

```
URL = "jdbc:mysql://localhost:3306/tienda"
USUARIO = "root"
PASSWORD = ""
```

Escribe un programa que haga **dos cosas**:

- **(a)** Cree la tabla empresa con las columnas id (INT, AUTO_INCREMENT, PRIMARY KEY), nombre (VARCHAR(100) NOT NULL) y tipo (VARCHAR(100) NOT NULL), usando un Statement y execute(...).
- **(b)** Inserte una empresa (nombre "Abogada Spain", tipo "Juridico") con un PreparedStatement usando parámetros ? y executeUpdate(). Muestra un mensaje si se ha insertado.

Gestiona SQLException con try-catch. Fíjate en tus clases InsertarTabla e InsertarDatos.

Tu respuesta:

RA9

Tema 11 – JDBC | Consultar (SELECT) y actualizar (UPDATE)

Sobre la misma tabla empresa(id, nombre, tipo) de la BD tienda, escribe un programa que haga **dos cosas**:

- **(a)** Ejecute SELECT * FROM empresa con un PreparedStatement, recorra el ResultSet con while(rs.next()) y muestre id, nombre y tipo alineados con System.out.printf.
- **(b)** Actualice con un UPDATE el campo tipo de la empresa cuyo nombre sea "Abogada Spain", usando PreparedStatement con parámetros ?.

Formato sugerido para el printf (como en tu clase Select):

```
System.out.printf("%-4d | %-20s | %-20s\n", id, nombre, tipo);
```

Gestiona SQLException. Fíjate en tus clases Select y Update.

Tu respuesta:

Soluciones

Bloque 1 – UT2

RA1 – Precio con IVA

```
import java.util.Scanner;
public class PrecioIVA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Precio: ");
        double precio = sc.nextDouble();
        System.out.print("Unidades: ");
        int unidades = sc.nextInt();
        double total = precio * unidades;
        double conIva = total * 1.21;
        System.out.println("Precio final (IVA incl.): " + conIva + " euros");
        sc.close();
    }
}
```

RA3 – ¿Qué se muestra por pantalla?

Los **impares** restan 1 a la suma y los **pares** suman el valor de i:

```
i=1 suma=-1
i=2 suma=1
i=3 suma=0
i=4 suma=4
i=5 suma=3
i=6 suma=9
Total: 9
```

Bloque 2 – UT3

RA2 – Clase Producto básica

```
public class Producto {
    String nombre;
    double precio;
    void mostrar() {
        System.out.println(nombre + " -> " + precio + " euros");
    }
}

public class Main {
    public static void main(String[] args) {
        Producto p1 = new Producto();
        p1.nombre = "Boli"; p1.precio = 1.20;
        Producto p2 = new Producto();
        p2.nombre = "Cuaderno"; p2.precio = 3.50;
        p1.mostrar(); p2.mostrar();
    }
}
```

RA4 – Producto con constructor, encapsulación y toString

```

public class Producto {
    private String nombre;
    private double precio;
    public Producto(String nombre, double precio) {
        this.nombre = nombre; this.precio = precio;
    }
    public String getNombre() { return nombre; }
    public void setNombre(String n) { this.nombre = n; }
    public double getPrecio() { return precio; }
    public void setPrecio(double p) { this.precio = p; }
    public void aplicarDescuento(double pct) {
        precio = precio - (precio * pct / 100);
    }
    @Override
    public String toString() {
        return "Producto: " + nombre + " - " + precio + " euros";
    }
    public static void main(String[] args) {
        Producto p = new Producto("Cuaderno", 3.50);
        p.aplicarDescuento(10);
        System.out.println(p);
    }
}

```

Bloque 3 – UT10 Ficheros

RA5 – Leer Biblioteca.txt en un ArrayList

```

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
public class LeerBiblioteca {
    public static void main(String[] args) {
        ArrayList<String> titulos = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(
            new FileReader("Biblioteca.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                String[] partes = linea.split(":");
                titulos.add(partes[0]);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
        System.out.println("Total titulos: " + titulos.size());
        for (String t : titulos) {
            System.out.println(t);
        }
    }
}

```

Bloque 4 – UT6/UT9 Colecciones de String

RA6 – Mejora el código (colecciones de String)

Error 1 (no compila): `n.charAt(0)` devuelve un `char`, no un `String`; no se puede comparar con "A". Hay que usar comillas simples: `== 'A'`.

Error 2 (excepción): `i <= nombres.size()` se sale del índice (`IndexOutOfBoundsException`). Debe ser `i < nombres.size()`.

```
for (int i = 0; i < nombres.size(); i++) {
    String n = nombres.get(i);
    if (n.charAt(0) == 'A') {
        System.out.println(n.toUpperCase());
    }
}
// Salida: ALBA
```

Bloque 5 – UT7/UT8 Excepciones + POO avanzada

RA7 – Clase abstracta + herencia + excepción

```
abstract class Figura { abstract double area(); }

class Circulo extends Figura {
    double radio;
    Circulo(double r) { this.radio = r; }
    double area() { return Math.PI * radio * radio; }
}

class Rectangulo extends Figura {
    double base, altura;
    Rectangulo(double b, double h) { base = b; altura = h; }
    double area() { return base * altura; }
}

public class Main {
    public static void main(String[] args) {
        String entrada = "hola";
        try {
            double r = Double.parseDouble(entrada);
            Circulo c = new Circulo(r);
            System.out.println("Area: " + c.area());
        } catch (NumberFormatException e) {
            System.out.println("Radio no valido");
        }
    }
}
```

Bloque 6 – Tema 11 JDBC

RA8 – Crear tabla empresa e insertar una fila

```

import java.sql.*;
public class CrearEInsertar {
    static final String URL = "jdbc:mysql://localhost:3306/tienda";
    static final String USUARIO = "root";
    static final String PASSWORD = "";
    public static void main(String[] args) {
        String crear = "CREATE TABLE empresa ("
            + "id INT AUTO_INCREMENT PRIMARY KEY, "
            + "nombre VARCHAR(100) NOT NULL, "
            + "tipo VARCHAR(100) NOT NULL)";
        String insertar = "INSERT INTO empresa (nombre, tipo) VALUES (?, ?)";
        try (Connection con = DriverManager.getConnection(URL, USUARIO, PASSWORD);
            Statement st = con.createStatement()) {
            st.execute(crear);
            System.out.println("Tabla creada");
            try (PreparedStatement ps = con.prepareStatement(insertar)) {
                ps.setString(1, "Abogada Spain");
                ps.setString(2, "Juridico");
                int filas = ps.executeUpdate();
                if (filas > 0)
                    System.out.println("Empresa insertada correctamente");
            }
        } catch (SQLException e) {
            System.err.println("Error de SQL: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

RA9 – SELECT con printf y UPDATE con PreparedStatement

```

import java.sql.*;
public class ConsultarActualizar {
    static final String URL = "jdbc:mysql://localhost:3306/tienda";
    static final String USUARIO = "root";
    static final String PASSWORD = "";
    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USUARIO, PASSWORD)) {
            // (a) SELECT
            String sel = "SELECT * FROM empresa";
            PreparedStatement pSel = con.prepareStatement(sel);
            ResultSet rst = pSel.executeQuery();
            while (rst.next()) {
                int id = rst.getInt("id");
                String nombre = rst.getString("nombre");
                String tipo = rst.getString("tipo");
                System.out.printf("%-4d | %-20s | %-20s\n", id, nombre, tipo);
            }
            // (b) UPDATE
            String upd = "UPDATE empresa SET tipo = ? WHERE nombre = ?";
            PreparedStatement pUpd = con.prepareStatement(upd);
            pUpd.setString(1, "Consultoria");
            pUpd.setString(2, "Abogada Spain");
            pUpd.executeUpdate();
            System.out.println("Empresa actualizada");
        } catch (SQLException e) {
            System.out.println(e.getMessage());
        }
    }
}

```